

Generative AI with Diffusion

An introduction to Diffusion Models

Politecnico di Milano | 6/05/2026

Vincenzo Pomponi, PhD Student

*Automation Robotics and Machines Laboratory (ARM)
Institute of System and Technologies for Sustainable Production (ISTePS)*

Welcome 😊



Feel free to contact me if you have any questions or curiosity 😊
vincenzo.pomponi@supsi.ch

Generative AI



Artistic Images



Realistic Videos



Draft Texts

These systems are “**creative**”
=
They are able to generate new objects.

DALL-E



DALL-E 3

Tiny potato kings wearing majestic crowns, sitting on thrones, overseeing their vast potato kingdom filled with potato subjects and potato castles.

[1]: <https://openai.com/index/dall-e-3/>

DALL-E



DALL-E 3

A 3D render of a coffee mug placed on a window sill during a stormy day. The storm outside the window is reflected in the coffee, with miniature lightning bolts and turbulent waves seen inside the mug. The room is dimly lit, adding to the dramatic atmosphere.

[1]: <https://openai.com/index/dall-e-3/>

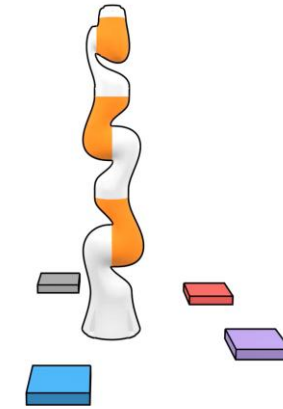
State of the Art Models



Stable Diffusion [2]
DALEE [5]



OpenAI Sora [3]
Meta MovieGen [6]



Diffusion Policy [4]
Motion Planning Diffusion [7]

Most SOTA generative AI models for images/videos/robotics: Diffusion Models

Lectures overview

- Lecture 1 (today):
 - From generation to sampling
 - Diffusion Models introduction
 - Forward Diffusion Process (closed form solution derivation)
 - Reverse Diffusion Process (training loss derivation)
- Lecture 2 (13/05/2026):
 - Recap of Lecture 1
 - DDPM vs DDIM
 - Diffusion Model architectures
 - Practical Python coding

Today overview

- From generation to sampling:
 - How to formalize the concept of generation?
- Diffusion Models introduction
 - Definition of diffusion models
- Forward Diffusion Process:
 - Modelling as a conditioned Gaussian distribution
 - Closed-form solution derivation
- Reverse Diffusion Process:
 - Intractability
 - Closed form solution if conditioned on x_0
 - Training loss derivation

Section 1: **From Generation to Sampling**

Goal: Formalize what it means to “generate” something.

Input Data

Images:

- Height H and Width W
- 3 color channels (RGB)



$$\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$$

Videos:

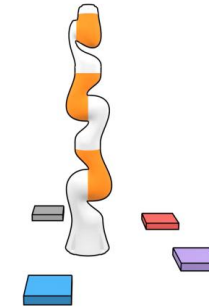
- T time frames
- Each frame is an image



$$\mathbf{x} \in \mathbb{R}^{T \times H \times W \times 3}$$

Robot trajectories:

- N robot configurations
- K DOFs + 1 gripper



$$\mathbf{x} \in \mathbb{R}^{N \times (K+1)}$$

We represent the objects we want to generate as vectors:

$$\mathbf{x} \in \mathbb{R}^d$$

Generation

Image of a dog



Generation

Image of a dog



Generation

Image of a dog

Wrong!



Not Good!



Perfect!



Can we formalize this?

Generation

Image of a dog

0.01



0.3



0.96



Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of X : $x \in \mathbb{R}^d$

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of $X: x \in \mathbb{R}^d$
- Data distribution: distribution of objects that we want to generate p_{data}

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of $X: x \in \mathbb{R}^d$
- Data distribution: distribution of objects that we want to generate p_{data}
- (Continuous) Probability Density Function (PDF): $p_{data}: \mathbb{R}^d \rightarrow \mathbb{R}_{>0}$

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of X : $x \in \mathbb{R}^d$
- Data distribution: distribution of objects that we want to generate p_{data}
- (Continuous) Probability Density Function (PDF): $p_{data}: \mathbb{R}^d \rightarrow \mathbb{R}_{>0}$
- d-dimensional isotropic Gaussian: $\mathcal{N}(x; \mu, \sigma^2 I) = (2 \pi \sigma^2)^{-\frac{d}{2}} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right)$

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of $X: x \in \mathbb{R}^d$
- Data distribution: distribution of objects that we want to generate p_{data}
- (Continuous) Probability Density Function (PDF): $p_{data}: \mathbb{R}^d \rightarrow \mathbb{R}_{>0}$
- d-dimensional isotropic Gaussian: $\mathcal{N}(x; \mu, \sigma^2 I) = (2 \pi \sigma^2)^{-\frac{d}{2}} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right)$
- Bayes' theorem: $P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$

Definitions

- Random Variable (RV): $X: \Omega \rightarrow \mathbb{R}^d$
- Realization (outcome) of $X: x \in \mathbb{R}^d$
- Data distribution: distribution of objects that we want to generate p_{data}
- (Continuous) Probability Density Function (PDF): $p_{data}: \mathbb{R}^d \rightarrow \mathbb{R}_{>0}$
- d-dimensional isotropic Gaussian: $\mathcal{N}(x; \mu, \sigma^2 I) = (2 \pi \sigma^2)^{-\frac{d}{2}} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right)$
- Bayes' theorem: $P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$
- Expectation: $\mathbb{E}_{p(x)}[f(x)] = \int f(x)p(x)dx$

Generation

Generation as sampling from the data distribution

Data distribution: distribution of objects we want to generate:

 p_{data}

Probability density:

$$p_{\text{data}} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0},$$
$$\mathbf{x} \mapsto p_{\text{data}}(\mathbf{x})$$

Generation means sampling from the data distribution:

$$\mathbf{x} \sim p_{\text{data}}$$



$$\mathbf{x} =$$



Summary

- **Objects to generate:** We focus on vectors x representing data objects (e.g. images, videos).
- **Data distribution:** Distribution that assigns higher values to objects that we consider “good”.
- **Generation as sampling:** generate an object = sampling from the data distribution.
- **Dataset:** Finite number of samples from the data distribution used for training.
- **Conditional Generation:** Condition on label y and sample from the conditional data distribution
- **Generative Model:** Train a model to transform samples from a simple (e.g., Gaussian) distribution into the data distribution.

Section 2: Diffusion Models

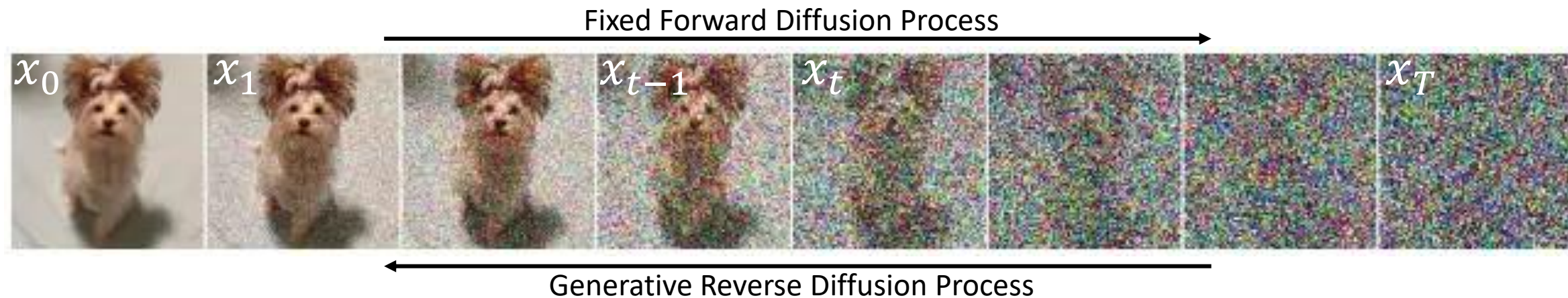
Goal: Define what are diffusion models

Diffusion Models

A Diffusion Model learns how to turn random noise into realistic data by gradually reversing a process that adds noise to real data.

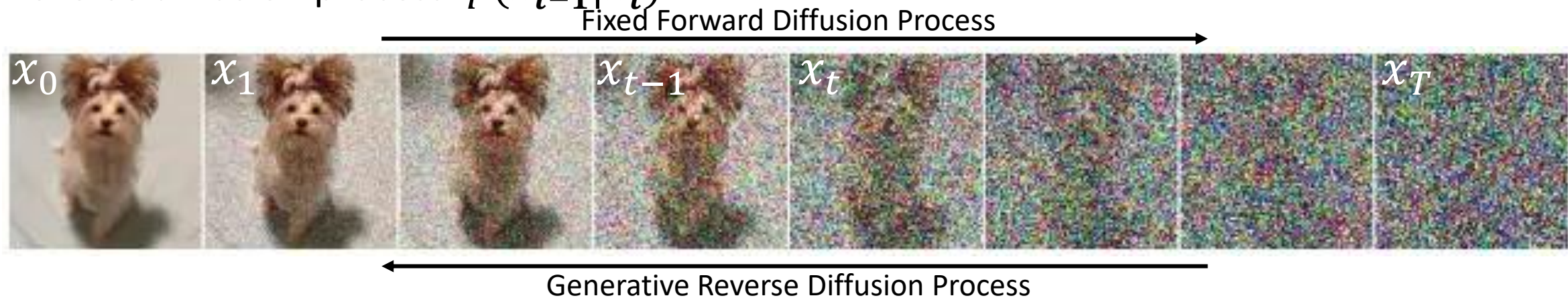
Diffusion Models work in two main steps:

- 1. Forward Diffusion Process:** gradually add noise to real data (e.g. images) until it becomes completely random, like pure Gaussian noise.
- 2. Reverse Diffusion Process:** the model learns how to reverse this process.



Variables

- Image: x_0
- Noisy image at timestep t : x_t
- T : diffusion steps
- Random noise (noisy image at timestep T): $x_T \sim \mathcal{N}(0, I)$
- Forward diffusion process: $q(x_t|x_{t-1})$
- Reverse diffusion process: $p(x_{t-1}|x_t)$



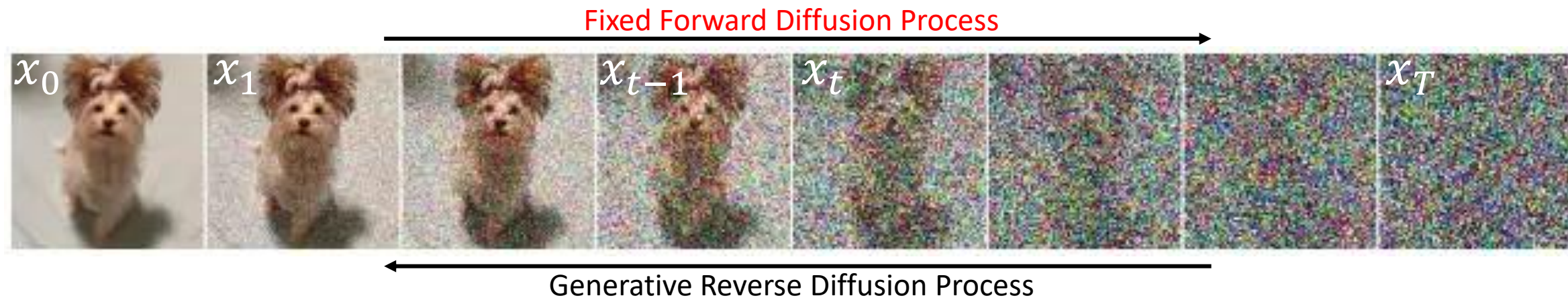
Section 2a: **Forward Diffusion Process**

Goal: mathematically define the forward diffusion process

Forward Diffusion Process

The forward diffusion process is defined as a Markov process that gradually perturbs the data by adding Gaussian noise over T discrete time steps, generating a sequence of latent variables $(x_1, x_2, \dots, x_t)$.

The step sizes are controlled by a variance schedule: $\{\beta_t \in (0, 1)\}_{t=1}^T$.



Forward Diffusion Process

The forward diffusion process, defined as:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

Corresponds to a Gaussian distribution with mean $\mu = \sqrt{1 - \beta_t} \mathbf{x}_{t-1}$ and covariance $\Sigma = \beta_t \mathbf{I}$.

Here, $\boldsymbol{\mu} \in \mathbb{R}^d$ and $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$.

The variance parameter β_t is a scalar and $\mathbf{I}$ denotes the $d \times d$ identity matrix.

The previous formulation describes the sampling of $\mathbf{x}_t$ from a Gaussian distribution **conditioned** on $\mathbf{x}_{t-1}$.

Reparameterization trick

If $x \sim \mathcal{N}(\mu, \Sigma)$, then it can be written as:

$$x = \mu + \Sigma^{1/2} \epsilon$$

With $\epsilon \sim \mathcal{N}(0, I)$.

This formulation isolates the stochastic component ϵ , making the remaining transformation **deterministic and differentiable**, which is essential for gradient-based optimization.

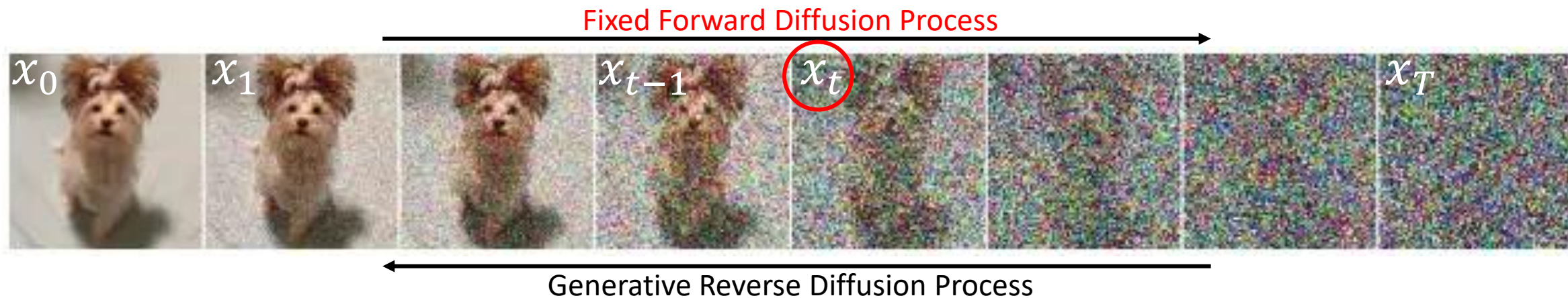
Reparameterization trick

In our setting, we obtain:

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \boldsymbol{\epsilon}$$

With $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

This equation describes a **single** diffusion step.



Markov property

Each diffusion step depends only on the previous one and it is independent from the others.

Therefore, we can apply the chain rule of probability, obtaining:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

If $T \rightarrow \infty$, the distribution of $\mathbf{x}_T$ approaches an isotropic Gaussian distribution.

Forward Diffusion Process

Let us define:

$$\alpha_t = 1 - \beta_t$$

Then, we can express the formulation of the noisy sample at diffusion step t in a closed form:

$$\begin{aligned} \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\ \mathbf{x}_{t-1} &= \sqrt{\alpha_t} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-2} \\ \mathbf{x}_{t-2} &= \sqrt{\alpha_t} \mathbf{x}_{t-3} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-3} \end{aligned}$$

Forward Diffusion Process

Substituting, we will obtain:

$$\mathbf{x}_t = \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})} \boldsymbol{\epsilon}_{t-2} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}$$

Since a linear combination of independent Gaussian variables is still Gaussian:

$$a\boldsymbol{\epsilon}_1 + b\boldsymbol{\epsilon}_2 = \sqrt{a^2 + b^2} \boldsymbol{\epsilon}, \quad \text{with } \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$

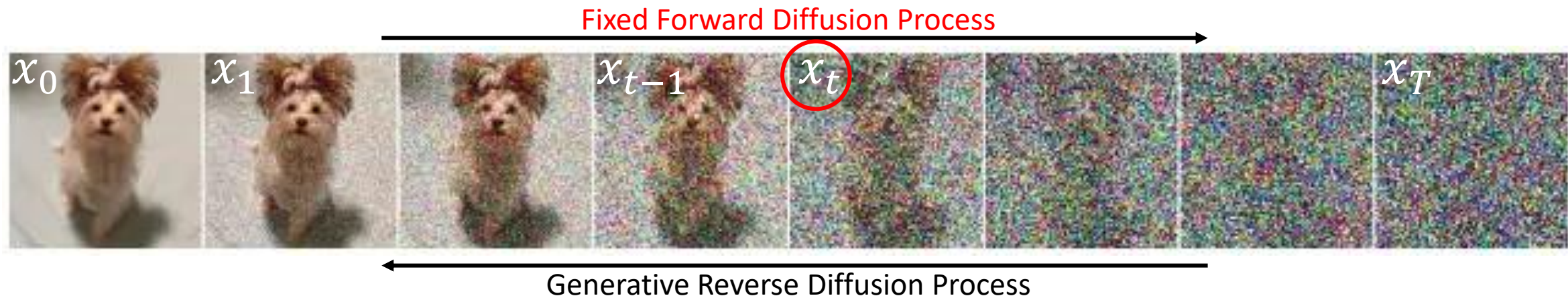
Finally:

$$\mathbf{x}_t = \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{(1 - \alpha_t \alpha_{t-1})} \boldsymbol{\epsilon}$$

Forward Diffusion Process

By recursively applying the previous procedure:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$
$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$



Forward Diffusion Process

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$$

The previous expression shows that:

- $\sqrt{\bar{\alpha}_t}$ controls how much of the original signal $\mathbf{x}_0$ is preserved.
- $\sqrt{1 - \bar{\alpha}_t}$ determines the amount of accumulated noise.

Equivalently, we can write the closed-form equation as:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

This means that, given a noise schedule $\{\beta_t \in (0, 1)\}_{t=1}^T$, the noisy sample at any diffusion step can be computed in closed form using the equation above.

Stochastic Langevin Dynamics

In the context of Diffusion Models, one way to model diffusion processes consists in progressively perturbing the data by adding noise at multiple levels, thereby constructing a family of increasingly noisy data distributions.

Starting from the original data distribution, each perturbation step produces a new distribution in which the data becomes gradually less structured and eventually approaches pure noise.

Langevin Dynamics is a method to generate samples from a probability distribution using two components:

- A deterministic step: moves toward more likely data.
- A random step: adds noise to keep diversity.

Stochastic Langevin Dynamics

The update rule is:

$$x_{k+1} = x_k + \eta \nabla_x \log p(x_k) + \sqrt{2\eta} z_k, \quad z_k \sim \mathcal{N}(0, 1)$$

Where x_k is current sample, η step size, $\nabla_x \log p(x_k)$ is the score (direction toward high-density regions), and z_k is the Gaussian noise.

Example: Think of a particle moving in a landscape. The landscape is the probability distribution, high regions are likely data and low regions are unlikely data. At each step, the particle moves uphill (toward more realistic data), but also jitters randomly (to explore different possibilities).

Summary

- The forward diffusion process is **FIXED** and controlled by the variance schedule β_t .
- **Markov property**
- **Reparameterization trick** makes each forward transformation deterministic and differentiable.
- Each noisy sample $\mathbf{x}_t$ at any diffusion step can be computed in **closed form** using:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$
$$\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$$

Section 2b:

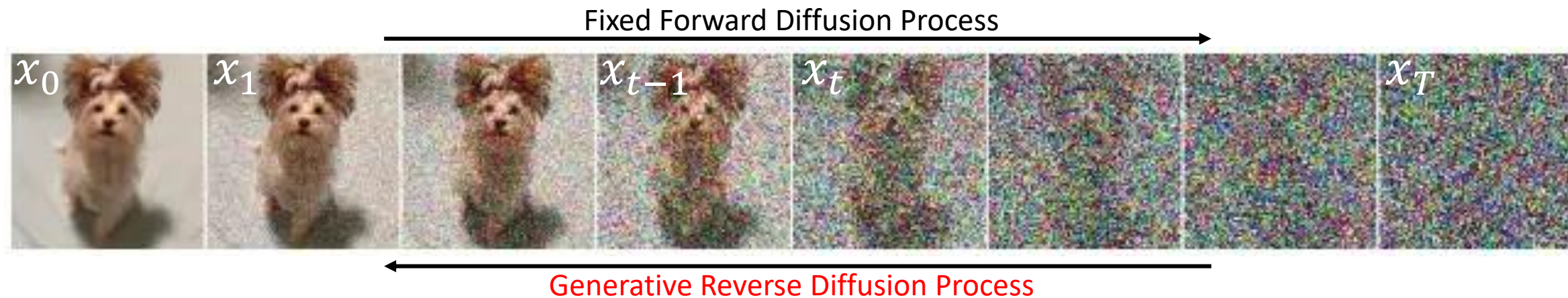
Reverse Diffusion Process

Goal: derive the loss function used for training diffusion models

Reverse Diffusion Process

The reverse diffusion process aims to invert the forward noising procedure by transforming a simple Gaussian noise distribution into the data distribution through a sequence of denoising steps.

Starting from a noise sample $x_T \sim \mathcal{N}(0, I)$, the goal is to progressively recover a structured data sample x_0 by iteratively reconstructing the vector x_{t-1} from x_t .



Reverse Diffusion Process

If we can reverse the above forward process and sample directly from:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

We will be able to recreate the true sample from a Gaussian noise input $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.

Unfortunately, we cannot easily estimate these conditional probabilities because the entire dataset (all possible data samples, such as images of dogs) is needed.

Reverse Diffusion Process

We can demonstrate that the reverse conditional distribution is intractable:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$

We have that both $q(\mathbf{x}_{t-1})$ and $q(\mathbf{x}_t)$ implicitly depend on $q(\mathbf{x}_0)$ which is unknown and intractable.

Consequently, $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also intractable!

$q(\mathbf{x}_0)$ denotes the true (unknown) data distribution from which the training samples are generated and we only can observe samples from it.

Reverse Diffusion Process

In contrast:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Using the Markov property: $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1})$

The term $q(\mathbf{x}_t|\mathbf{x}_0)$ is constant as does not depends on $\mathbf{x}_{t-1}$ which is the random variable we are interested in modelling.

Therefore:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \propto q(\mathbf{x}_t|\mathbf{x}_{t-1}) q(\mathbf{x}_{t-1}|\mathbf{x}_0)$$

Tractable means that we can compute it analytically and we are going to see how to compute it in the following slides.

Reverse Diffusion Process

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}) \propto \exp\left(-\frac{1}{2(1 - \alpha_t)} \|\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1}\|^2\right)$$
$$q(\mathbf{x}_{t-1} | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) \mathbf{I}) \propto \exp\left(-\frac{1}{2(1 - \bar{\alpha}_{t-1})} \|\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0\|^2\right)$$

Combining the two Gaussians, we obtain simple formulation for the mean $\tilde{\mu}_t$ and covariance $\tilde{\beta}_t \mathbf{I}$ of the resulting Gaussian distribution:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right)$$
$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$$

Reverse Diffusion Process

We have demonstrated that the reverse conditional distribution is tractable when conditioned on $\mathbf{x}_0$ and can be approximated with a Gaussian distribution with mean $\tilde{\boldsymbol{\mu}}_t$ and covariance $\tilde{\boldsymbol{\beta}}_t \mathbf{I}$:

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\boldsymbol{\beta}}_t \mathbf{I})$$

Reverse Diffusion Process

At generation time, the original data point $\mathbf{x}_0$ is unknown and the intermediate (latent) variables $\mathbf{x}_{1:T-1}$ cannot be observed.

Therefore, we cannot use $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$.

To address this limitation, we introduce a parameterized model:

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Which aims to approximate the true reverse dynamics without requiring access to $\mathbf{x}_0$.

The goal of this model is to generate realistic data samples, and to quantify this, we study the likelihood of the data under the model, denoted by:

$$p_{\theta}(\mathbf{x}_0)$$

Reverse Diffusion Process

This quantity is the **marginal** likelihood of the data, i.e., the probability density that the model assigns to a data point $\mathbf{x}_0$, irrespective of the latent trajectory that generated it.

Formally, it is obtained by integrating (marginalizing) out all latent variables:

$$p_{\theta}(\mathbf{x}_0) = \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

Reverse Diffusion Process



INTRACTABLE



$$p_{\theta}(x_0) = \int p_{\theta}(x_{0:T}) dx_{1:T}$$

Reverse Diffusion Process

Rather than working directly with the joint distribution, we consider its logarithm. This transformation provides two key advantages:

- Numerical stability.
- Sums instead of products
- Maximizing the likelihood is equivalent to maximizing the log-likelihood.

$$\log p_{\theta}(\mathbf{x}_0) = \log \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

Reverse Diffusion Process

As we said, the integral is intractable.

To overcome this problem, we introduce an auxiliary distribution which we know it is tractable:

THE FORWARD PROCESS

$$\log p_{\theta}(\mathbf{x}_0) = \log \int q(\mathbf{x}_{1:T}|\mathbf{x}_0) \frac{p_{\theta}(\mathbf{x}_{0:T})d\mathbf{x}_{1:T}}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} d\mathbf{x}_{1:T}$$

Reverse Diffusion Process

To make the log-likelihood tractable, we introduce the concept of *expectation* over a known distribution.

In particular, the expectation allows us to rewrite the log-likelihood as an average of log-probabilities, where each trajectory contributes according to how likely it is under the forward diffusion process.

$$\log p_{\theta}(\mathbf{x}_0) = \log \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right]$$

Reverse Diffusion Process

To further transform the log-likelihood into a tractable objective, we apply **Jensen's inequality**.

Jensen's inequality states that the logarithm of an expectation is greater than or equal to the expectation of the logarithm.

This allows us to move the logarithm inside the expectation, which is crucial for tractability.

$$\log p_{\theta}(\mathbf{x}_0) = \log \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right]$$

Reverse Diffusion Process

After applying Jensen's inequality, we obtain a new objective called the **Evidence Lower Bound (ELBO)**.

The **ELBO** is a tractable lower bound on the log-likelihood of the data.

Instead of maximizing the true log-likelihood—which is intractable—we maximize this lower bound.

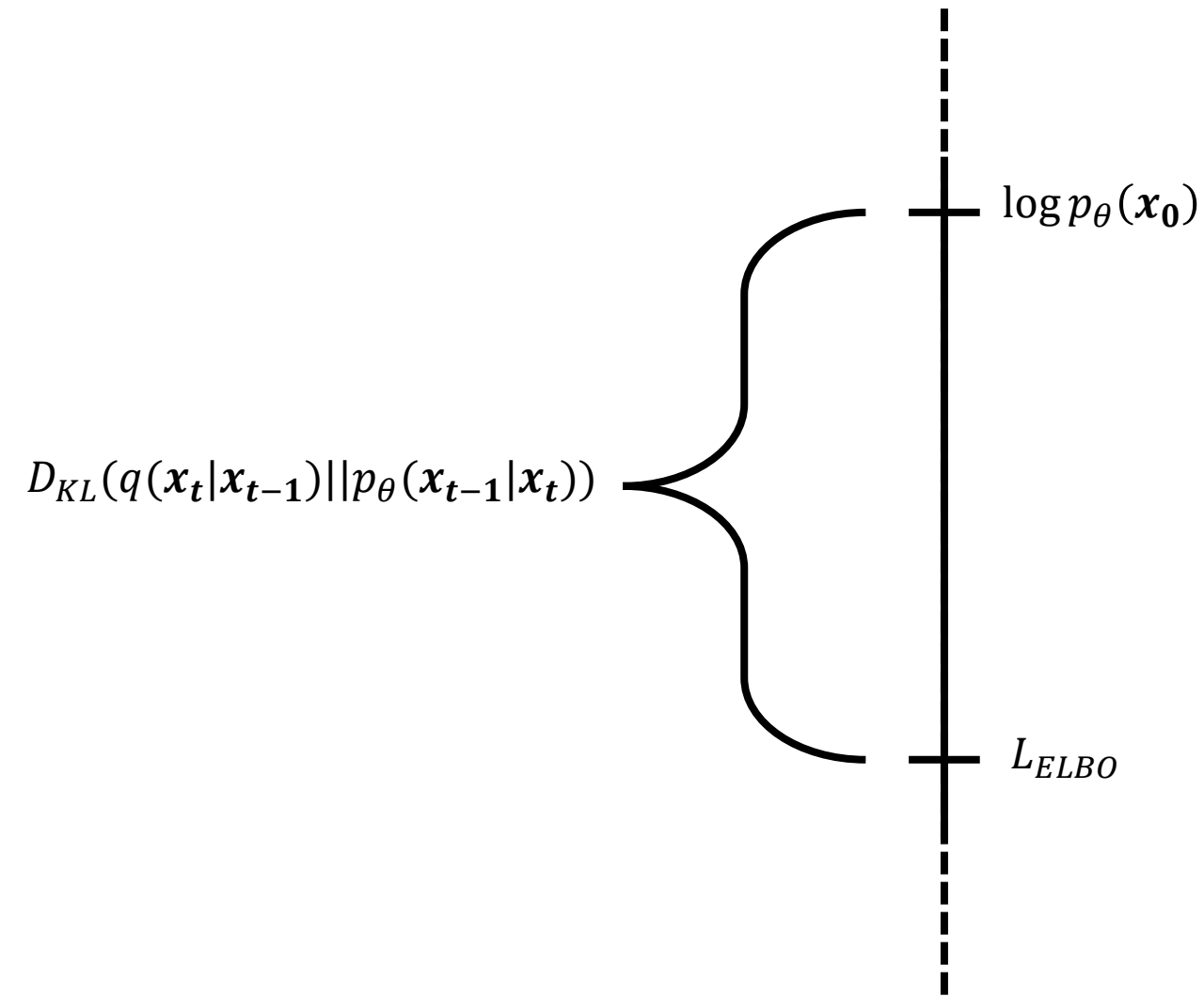
$$L_{ELBO} = \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} [\log p_{\theta}(\mathbf{x}_{0:T}) - \log q(\mathbf{x}_{1:T}|\mathbf{x}_0)]$$

Reverse Diffusion Process

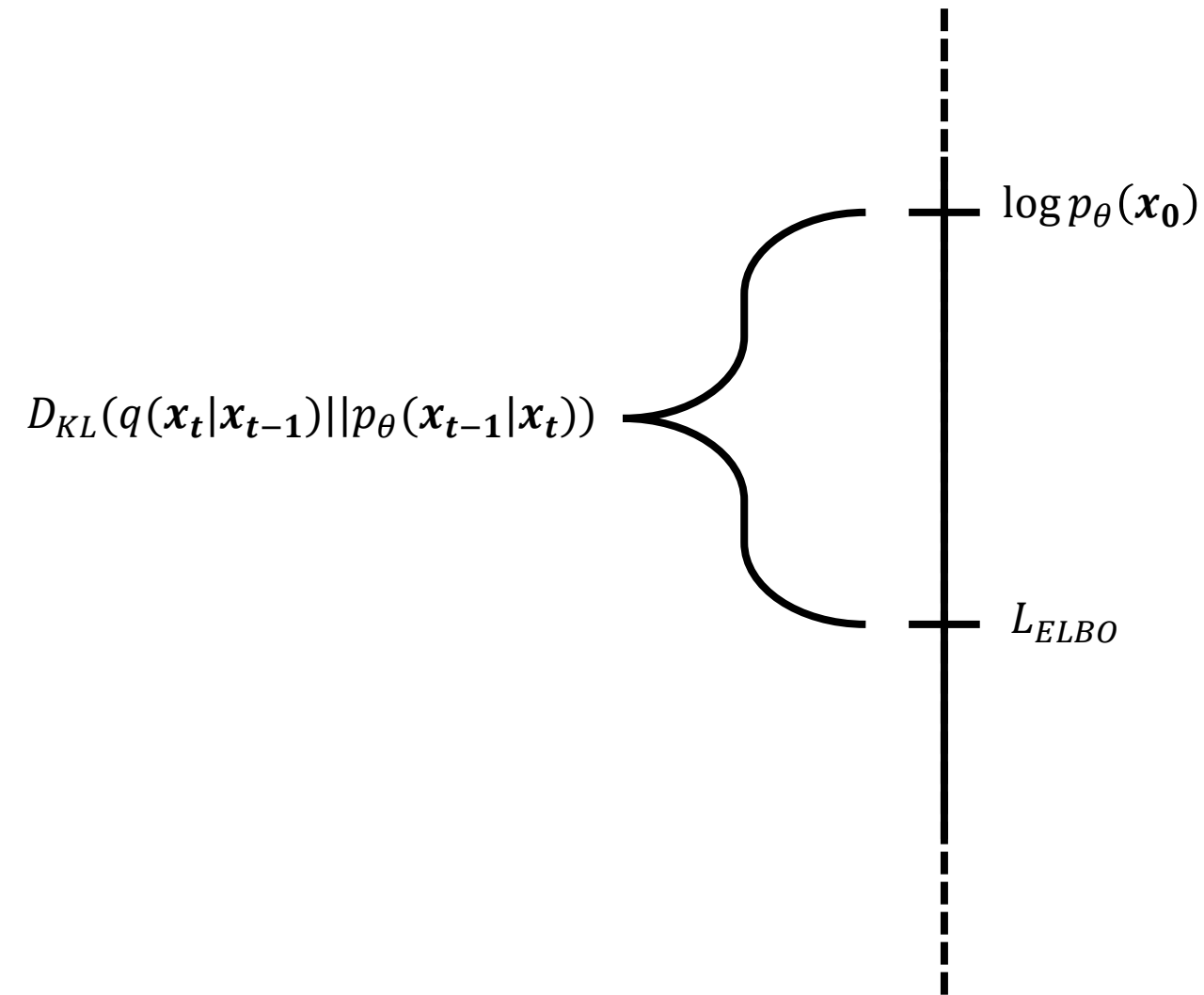
Therefore, we obtained:

$$L_{ELBO} \leq \log p_{\theta}(x_0)$$

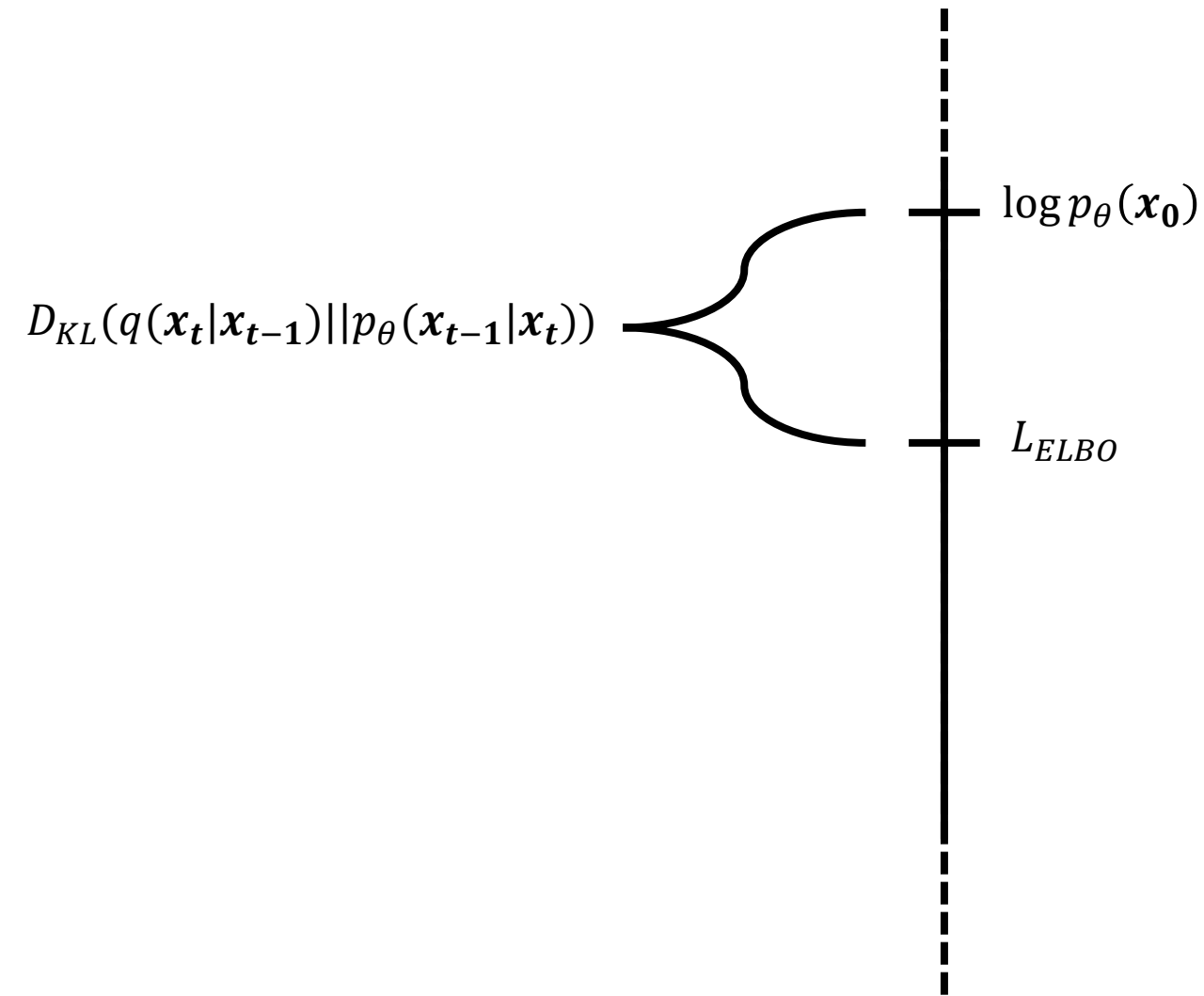
Reverse Diffusion Process



Reverse Diffusion Process



Reverse Diffusion Process



Reverse Diffusion Process

$$L_{ELBO} \uparrow \quad D_{KL} \rightarrow 0, \quad L_{ELBO} \approx \log p_{\theta}(x_0)$$

Reverse Diffusion Process

Recall that:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$
$$\log p_{\theta}(\mathbf{x}_{0:T}) = \log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Reverse Diffusion Process

In the same way, we can expand the forward process:

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

$$\log q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \sum_{t=1}^T \log q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

$$L_{ELBO} = \mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) - \sum_{t=1}^T \log q(\mathbf{x}_t|\mathbf{x}_{t-1}) \right]$$

Reverse Diffusion Process

Remembering the Bayes' rule to the reverse conditional probability:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)}$$

Applying the Markov property: $q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t|\mathbf{x}_{t-1})$

Rearranging it:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}$$

Taking the logarithm and substituting into the L_{ELBO}

$$L_{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}_T)] + \sum_{t=1}^T \mathbb{E}_q[\log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log q(\mathbf{x}_t|\mathbf{x}_0) + \log q(\mathbf{x}_{t-1}|\mathbf{x}_0)]$$

Reverse Diffusion Process

$$L_{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}_T)] - \sum_{t=1}^T \mathbb{E}_q[\log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) + \log q(\mathbf{x}_t|\mathbf{x}_0) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_0)]$$

Let us analyze the last two terms:

$$\sum_{t=1}^T [\log q(\mathbf{x}_t|\mathbf{x}_0) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_0)]$$

For $t = 1$: $\log q(\mathbf{x}_1|\mathbf{x}_0) - \log q(\mathbf{x}_0|\mathbf{x}_0)$

For $t = 2$: $\log q(\mathbf{x}_2|\mathbf{x}_0) - \log q(\mathbf{x}_1|\mathbf{x}_0)$

For $t = 3$: $\log q(\mathbf{x}_3|\mathbf{x}_0) - \log q(\mathbf{x}_2|\mathbf{x}_0)$

TELESCOPIC CANCELLATION!!

Reverse Diffusion Process

$$L_{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}_T)] - \sum_{t=1}^T \mathbb{E}_q[\log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) + \log q(\mathbf{x}_t|\mathbf{x}_0) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_0)]$$

We obtain: $\log q(\mathbf{x}_T|\mathbf{x}_0) - \log q(\mathbf{x}_0|\mathbf{x}_0)$

But $q(\mathbf{x}_0|\mathbf{x}_0) = 1$ and $\log q(\mathbf{x}_0|\mathbf{x}_0) = 0$

Therefore, the ELBO then becomes:

$$L_{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}_T)] - \sum_{t=1}^T \mathbb{E}_q[\log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)] + \mathbb{E}_q[\log q(\mathbf{x}_T|\mathbf{x}_0)]$$

Reverse Diffusion Process

$$L_{ELBO} = \mathbb{E}_q[\log p(\mathbf{x}_T)] + \sum_{t=1}^T \mathbb{E}_q[\log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) - \log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)] + \mathbb{E}_q[\log q(\mathbf{x}_T|\mathbf{x}_0)]$$

Now, we analyze the terms in green:

$$\sum_{t=1}^T \mathbb{E}_q[\log p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) - \log q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)]$$

For $t = 1$: $\log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \rightarrow$ **Reconstruction likelihood**

For $t \geq 2$: $D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) \rightarrow$ **KL divergences**

Reverse Diffusion Process

$$L_{ELBO} = \underbrace{\mathbb{E}_q[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]}_{L_0} + \underbrace{\mathbb{E}_q[D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))]}_{L_T} - \sum_{t=2}^T \underbrace{D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}}$$

Every KL term in L_{ELBO} (except for L_0) compares two Gaussian distributions and therefore they can be computed in closed form.

L_T is constant and can be ignored during training because q has no learnable parameters and $\mathbf{x}_T$ is a Gaussian noise.

Reverse Diffusion Process

$$L_{ELBO} = - \sum_{t=2}^T \mathbb{E} \left[D_{KL} \left(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) \right) \right] + \textit{CONSTANT}$$

Reverse Diffusion Process

The goal now is to maximize the ELBO, so that the $D_{KL}(q||p_\theta)$ will be minimized. In this way we are forcing the model posterior to match the forward process and to do so we train that model using the ELBO:

$$q(x_{t-1}|x_t, x_0) \approx p_\theta(x_{t-1}|x_t)$$

Reverse Diffusion Process

To recover the training loss, we focus on one single KL term:

$$D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))$$

Both distributions are Gaussian:

- True posterior: $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I})$
- Model: $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$

For simplicity, the true posterior and the model have the same covariance $\tilde{\beta}_t$.

Substituting, we obtain:

$$D_{KL} = \frac{1}{2\tilde{\beta}_t} \|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t)\|^2$$

Reverse Diffusion Process

But, instead predicting the whole noisy image, it is demonstrated that predicting the noise is more efficient [9].

The loss function, then, becomes:

$$L = ||\epsilon - \epsilon_{\theta}(x_t, t)||^2$$

Summary

- The true reverse process is intractable $\rightarrow$ it depends on the unknown data distribution.
- The true reverse process becomes tractable only when conditioned on $\mathbf{x}_0$.
- But $\mathbf{x}_0$ is not available at generation time.
- We introduced a learned model: $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. The goal is to approximate the true reverse dynamics with a neural network.
- We want to maximize the log likelihood of the data: $\log p_\theta(\mathbf{x}_0)$ but it is intractable.
- The solution is to optimize a tractable lower bound: the Evidence Lower Bound.
- We maximize the ELBO (or minimize the negative ELBO), as the D_{KL} will be reduced, and approximating the true reverse process $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ with our model $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$.
- Finally obtaining the formulation of the loss function for diffusion models:

$$L = \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2$$

References

The authors acknowledge the lecture notes from *MIT Course 6.S184: Generative AI with Stochastic Differential Equations* (<https://diffusion.csail.mit.edu/2026/index.html>) as a reference for this presentation.

- [1]: Matteo Meregalli Falerni, Vincenzo Pomponi, Hamid Reza Karimi, Matteo Lavit Nicora, Le Anh Dao, Matteo Malosio, & Loris Roveda (2024).
A framework for human–robot collaboration enhanced by preference learning and ergonomics. *Robotics and Computer-Integrated Manufacturing*, 89, 102781.
- [2]: Rombach, Robin, et al. "High-resolution image synthesis with latent diffusion models." *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022.
- [3]: Zheng, Zangwei, et al. "Open-sora: Democratizing efficient video production for all." *arXiv preprint arXiv:2412.20404* (2024).
- [4]: Chi, Cheng, et al. "Diffusion policy: Visuomotor policy learning via action diffusion." *The International Journal of Robotics Research* 44.10-11 (2025): 1684-1704.
- [5]: Ramesh, Aditya, et al. "Zero-shot text-to-image generation." *International conference on machine learning*. Pmlr, 2021.
- [6]: Polyak, Adam, et al. "Movie gen: A cast of media foundation models." *arXiv preprint arXiv:2410.13720* (2024).
- [7]: Carvalho, Joao, et al. "Motion planning diffusion: Learning and planning of robot motions with diffusion models." *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023.
- [8]: J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

Thanks for the attention



Feel free to contact me if you have any questions or curiosity 😊
vincenzo.pomponi@supsi.ch